

이산수학

제1장

수의 표현과 연산

1. 수의 체계
2. 수의 연산
3. 진법별 변환
4. 진법 간 변환
5. 진법별 사칙연산
6. 컴퓨터에서의 수의 표현과 연산

01001110 01010101 01001101

10110010 00101100 11010101

00010110 11001010 01011010

11101010 01010010 00110101

01011010 10101010 11001011

10010110 11010101 00101100

00110101 01010010 10101101

11001011 00101100 01010010

0x2A → 42 → 101010₂

강의 목차

01

수의 체계

자연수, 정수, 유리수, 실수, 복소수

02

수의 연산

사칙연산, 나눗셈 원리, 모듈러 산술

03

진법별 변환

2·8·16진법, 표기법, 자릿값

04

진법 간 변환

10진수 ↔ 타 진법 변환 알고리즘

05

진법별 사칙연산

2진 덧셈·뺄셈·곱셈·나눗셈

06

컴퓨터에서의 수의 표현

부호·보수·부동소수점·오버플로우

Section

01

수의 체계

자연수 · 정수 · 유리수 · 실수 · 복소수
수 체계의 확장 and 성질

개요

수 체계 (Number System)

C 복소수 (Complex)

R 실수 (Real)

Q 유리수 (Rational)

Z 정수 (Integer)

N 자연수 (Natural)

N $\subset$ **Z** $\subset$ **Q** $\subset$ **R** $\subset$ **C**

포함 관계

자연수 N

1, 2, 3, ... (셀 수 있음)

정수 Z

..., -2, -1, 0, 1, 2, ...

유리수 Q

p/q 형태 ($p, q \in \mathbb{Z}, q \neq 0$)

실수 R

유리수 + 무리수 ($\sqrt{2}, \pi$ 등)

복소수 C

$a+bi$ 형태 ($a, b \in \mathbb{R}$)

자연수($\mathbb{N}$)와 정수($\mathbb{Z}$)

자연수 $\mathbb{N} = \{1, 2, 3, 4, \dots\}$

- 덧셈과 곱셈에 대해 닫혀 있음
- 뺄셈: $1 - 2$ 는 $\mathbb{N}$ 에 속하지 않음 $\rightarrow \mathbb{Z}$ 로 확장
- 나눗셈: $3 \div 2$ 는 $\mathbb{N}$ 에 속하지 않음
- 정렬 원리(Well-Ordering): 공집합이 아닌 $\mathbb{N}$ 의 부분집합은 최솟값 존재

정수 $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$

- 덧셈, 뺄셈, 곱셈에 대해 닫혀 있음
- 항등원: 덧셈 $\rightarrow 0$, 곱셈 $\rightarrow 1$
- 역원: 덧셈 역원 존재 ($-a$), 곱셈 역원 일반적으로 $\mathbb{Z}$ 미포함
- 순서 관계: 전순서(total order) 성립



1-2 유리수와 무리수

유리수($\mathbb{Q}$)와 무리수

유리수 (Rational Number)

분수 p/q (p, q 는 정수, $q \neq 0$) 꼴로 나타낼 수 있는 수
 소수 표현: 유한소수 or 순환소수
 예) $1/2 = 0.5$, $1/3 = 0.333\dots$, $-7/4 = -1.75$

무리수 (Irrational Number)

유리수가 아닌 실수. 무한 비순환소수.
 예) $\sqrt{2} = 1.41421\dots$, $\pi = 3.14159\dots$, $e = 2.71828\dots$

실수 = 유리수 $\cup$ 무리수

유리수와 무리수는 서로소 (교집합 = $\emptyset$)
 실수 직선(real line)은 연속(continuous)

구분	유리수 $\mathbb{Q}$	무리수
표현	p/q 분수	분수 불가
소수	유한 or 순환	무한 비순환
덧셈 닫힘	✓	✗ ($\sqrt{2}+1$ 등)
예시	$1/2, -3, 0.75$	$\sqrt{2}, \pi, e, \sqrt{3}$

복소수 (Complex Number) $\mathbb{C}$

$$a + bi \quad (a, b \in \mathbb{R}, \quad i^2 = -1)$$

실수부 (Real Part)

$$\operatorname{Re}(z) = a$$

$$\text{예) } z = 3 + 4i \rightarrow \operatorname{Re}(z) = 3$$

허수부 (Imaginary Part)

$$\operatorname{Im}(z) = b$$

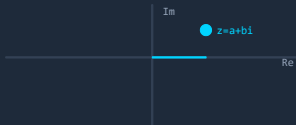
$$\text{예) } z = 3 + 4i \rightarrow \operatorname{Im}(z) = 4$$

켈레복소수

$$\bar{z} = a - bi$$

$$|z|^2 = z \cdot \bar{z} = a^2 + b^2$$

복소 평면 (Argand Diagram)



덧셈

$$(a+bi) + (c+di) = (a+c) + (b+d)i$$

뺄셈

$$(a+bi) - (c+di) = (a-c) + (b-d)i$$

곱셈

$$(a+bi)(c+di) = (ac-bd) + (ad+bc)i$$

나눗셈

$$(a+bi) \div (c+di) = (a+bi)(c-di) / (c^2+d^2)$$

수 체계의 대수적 성질

성질	N	Z	Q	R	C
덧셈 닫힘	✓	✓	✓	✓	✓
뺄셈 닫힘	X	✓	✓	✓	✓
곱셈 닫힘	✓	✓	✓	✓	✓
나눗셈 닫힘	X	X	✓ (0 제외)	✓ (0 제외)	✓ (0 제외)
덧셈 역원	X	✓	✓	✓	✓
곱셈 역원	X	X	✓ (0 제외)	✓ (0 제외)	✓ (0 제외)
순서 관계	✓	✓	✓	✓	X
완비성 (completeness)	X	X	X	✓	✓

✓ = 성립 X = 미성립 완비성: 실수는 '빈 곳 없이 채워진' 연속 집합

소수(Prime)와 합성수(Composite)

소수 (Prime Number)

1보다 큰 자연수 중 1과 자기 자신만으로 나누어지는 수

예) 2, 3, 5, 7, 11, 13, 17, 19, 23, ...

- 가장 작은 소수 = 2 (유일한 짝수 소수)
- 소수의 개수: 무한히 많음 (유클리드 증명)

합성수 (Composite Number)

1보다 큰 자연수 중 소수가 아닌 수

예) $4 = 2^2$, $6 = 2 \times 3$, $8 = 2^3$, $9 = 3^2$, $10 = 2 \times 5$

- 소인수분해(Unique Factorization): 모든 합성수는 소수의 곱으로 유일하게 나타낼 수 있음

// 소수 판별 알고리즘 (C언어)

```
int is_prime(int n) {
    if (n < 2) return 0;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) return 0;
    return 1; // 소수
}
```

// 시간복잡도: $O(\sqrt{n})$

에라토스테네스의 체 (Sieve of Eratosthenes)

2부터 N까지 소수를 빠르게 구하는 알고리즘

- 2의 배수 제거 → ② 3의 배수 제거 → ...
 - $\sqrt{N}$ 이하의 소수만 확인하면 됨
- 시간복잡도: $O(N \log \log N)$

최대공약수(GCD)와 최소공배수(LCM)

GCD (Greatest Common Divisor) 최대공약수

두 정수의 공약수 중 가장 큰 것

$$\text{gcd}(12, 18) = 6$$

$$\rightarrow 12 = 2^2 \times 3, 18 = 2 \times 3^2 \rightarrow \text{gcd} = 2^1 \times 3^1 = 6$$

LCM (Least Common Multiple) 최소공배수

두 정수의 공배수 중 가장 작은 것

$$\text{lcm}(12, 18) = 36$$

$$\text{관계식: } \text{lcm}(a, b) = a \times b / \text{gcd}(a, b)$$

```
// 유클리드 호제법 gcd(a, b)
```

```
int gcd(int a, int b) {
```

```
    while (b != 0) {
```

```
        int r = a % b;
```

```
        a = b; b = r;
```

```
    }
```

```
    return a;
```

```
}
```

```
// gcd(48, 18): 48%18=12 → 18%12=6 →
```

```
12%6=0 → 6
```

$$\text{gcd}(48, 18)$$

$$\rightarrow 48 = 2 \times 18 + 12$$

$$\rightarrow 18 = 1 \times 12 + 6$$

$$\rightarrow 12 = 2 \times 6 + 0$$

$$\therefore \text{gcd} = 6$$

$$\text{lcm}(48, 18) = 48 \times 18 / 6 = 144$$

절댓값과 나눗셈의 나머지

절댓값 $|a|$

$$|a| = a \quad (a \geq 0)$$

$$|a| = -a \quad (a < 0)$$

성질: $|ab| = |a||b|$, $|a+b| \leq |a| + |b|$ (삼각 부등식)

$$a = 17, \quad b = 5$$

$$q = a / b = 17 / 5 = 3 \quad (\text{정수 나눗셈})$$

$$r = a \% b = 17 \% 5 = 2$$

$$\text{검증: } 5 \times 3 + 2 = 17 \quad \checkmark$$

C언어: `int q = a / b;`
`int r = a % b;`

나눗셈 원리 (Division Algorithm)

정수 a 를 양의 정수 b 로 나누면

$$a = b \cdot q + r \quad (0 \leq r < b)$$

a : 피제수, b : 제수, q : 몫, r : 나머지

예) $17 \div 5 = 3 \dots 2 \rightarrow 17 = 5 \times 3 + 2$

음수의 나머지 (C언어 주의!)

수학적 나머지: 항상 ≥ 0

C에서 $-7 \% 3 = -1$ (부호는 피제수를 따름)

$\rightarrow$ 올바른 나머지: $((-7) \% 3 + 3) \% 3 = 2$

알고리즘 문제에서 음수 나머지 처리 필수!

모듈러 산술 (Modular Arithmetic)

$$a \equiv b \pmod{m} \iff m \mid (a - b)$$

덧셈

$$(a+b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$

뺄셈

$$(a-b) \bmod m = ((a \bmod m) - (b \bmod m) + m) \bmod m$$

곱셈

$$(a \times b) \bmod m = ((a \bmod m) \times (b \bmod m)) \bmod m$$

거듭제곱

$a^k \bmod m \rightarrow$ 빠른 거듭제곱(분할정복) 사용

```
// 빠른 거듭제곱  $a^k \bmod m$   
long long power(long long a, long long k, long long m) {  
    long long res = 1; a %= m;  
    while (k > 0) {  
        if (k & 1) res = res * a % m;  
        a = a * a % m; k >>= 1;  
    }  
    return res; //  $O(\log k)$   
}
```

SUMMARY

Section 01 핵심 정리

수 체계 포함 관계

$$\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$$

GCD / LCM

유클리드 호제법. $\text{lcm}(a,b) = a \times b / \text{gcd}(a,b)$

모듈러 산술

$$(a \text{ op } b) \bmod m = ((a \bmod m) \text{ op } (b \bmod m)) \bmod m$$

소수

1과 자신만의 약수. 에라토스테네스의 체 $O(N \log \log N)$

나눗셈 원리

$$a = b \cdot q + r, \quad 0 \leq r < b$$

음수 나머지

C언어에서 $((a \% m) + m) \% m$ 으로 처리

Section

02

수의 연산

사칙연산 · 연산 법칙 · 지수·로그
거듭제곱과 비트 연산 기초

사칙연산과 연산 법칙

법칙	덧셈 (+)	곱셈 (×)
교환 법칙	$a + b = b + a$	$a \times b = b \times a$
결합 법칙	$(a+b)+c = a+(b+c)$	$(a \times b) \times c = a \times (b \times c)$
항등원	$a + 0 = a$	$a \times 1 = a$
역원	$a + (-a) = 0$	$a \times (1/a) = 1 \quad (a \neq 0)$
분배 법칙	$a \times (b+c) = a \times b + a \times c$	$(a+b) \times c = a \times c + b \times c$

⚠ 뺄셈은 교환·결합 법칙 불성립: $5-3 \neq 3-5$

⚠ 나눗셈도 교환·결합 법칙 불성립: $6 \div 3 \neq 3 \div 6$

⚠ 0으로 나눗셈은 정의 불가 (Division by zero)

지수(Exponent)와 로그(Logarithm)

지수 법칙

$$a^m \times a^n = a^{m+n}$$

$$a^m \div a^n = a^{m-n}$$

$$(a^m)^n = a^{mn}$$

$$(ab)^n = a^n b^n$$

$$a^0 = 1 \quad (a \neq 0)$$

$$a^{-n} = 1/a^n$$

로그 법칙

$$\log_a(MN) = \log_a M + \log_a N$$

$$\log_a(M/N) = \log_a M - \log_a N$$

$$\log_a(M^k) = k \cdot \log_a M$$

$$\log_a b = \log_c b / \log_c a \quad (\text{밑 변환})$$

$$\ln x = \log_e x \quad (\text{자연로그})$$

$\log_2 n$

이진 탐색의 시간복잡도 단계 수

2^n

n비트로 표현 가능한 수의 개수

$n!$

순열의 수 (팩토리얼)

$\log_2 n!$

$\approx n \log_2 n$ (정렬 하한)

비트 연산 (Bitwise Operation)

연산자	기호	설명	예 (a=12, b=10)
AND	&	둘 다 1이면 1	$1100 \& 1010 = 1000 = 8$
OR		하나라도 1이면 1	$1100 1010 = 1110 = 14$
XOR	^	다르면 1	$1100 \wedge 1010 = 0110 = 6$
NOT	~	비트 반전	$\sim 1100 = \dots 0011 = -13$
LEFT SHIFT	<<	비트 왼쪽 이동 ($\times 2$)	$1 \ll 3 = 8$
RIGHT SHIFT	>>	비트 오른쪽 이동 ($\div 2$)	$8 \gg 1 = 4$

▶ $n \& 1 == 0 \rightarrow$ 짝수 판별

▶ $n \& (n-1) == 0 \rightarrow$ 2의 거듭제곱 판별

▶ $n | (1 \ll k) \rightarrow$ k번째 비트 세팅

▶ $n \& \sim(1 \ll k) \rightarrow$ k번째 비트 초기화

2-4 연산 우선순위

C언어 연산 우선순위

1 (최고)	() [] -> .	괄호, 배열, 멤버 접근
2	! ~ ++ -- (단항) * &	단항 연산자, 역참조
3	* / %	곱셈, 나눗셈, 나머지
4	+ -	덧셈, 뺄셈
5	<< >>	비트 시프트
6	< <= > >=	비교 연산
7	== !=	동등 비교
8-11	& ^ &&	비트/논리 연산 (낮은 우선순위 주의!)
12-15	= += -= ... ,	대입, 복합 대입, 쉼표

2-5 오버플로우

정수 오버플로우 (Integer Overflow)

int 범위 (32비트 부호 있음)

최솟값: $-2,147,483,648$ ($= -2^{31}$)

최댓값: $2,147,483,647$ ($= 2^{31} - 1$)

오버플로우 시: 부호 반전 (undefined behavior in C)

long long 범위 (64비트)

최솟값: $-9,223,372,036,854,775,808$ ($= -2^{63}$)

최댓값: $9,223,372,036,854,775,807$ ($= 2^{63} - 1$)

큰 수 연산 시 long long 사용 필수!

```
// 오버플로우 예시
int a = 2147483647;
a = a + 1; // → -2147483648 (오버플로우!)
```

```
// 안전한 덧셈 체크
if (a > INT_MAX - b) { /* overflow */ }
```

```
// 곱셈 시 long long 캐스팅
long long res = (long long)a * b;
```

Section

03

진법별 변환

2진법 · 8진법 · 16진법
자릿값의 원리와 표기법

진법 (Numeral System) 개요

BASE 2

이진법 (Binary)

숫자: 0, 1

접두사: 0b / 없음 용도: 컴퓨터 내부

BASE 10

십진법 (Decimal)

숫자: 0 ~ 9

접두사: 없음 용도: 일상생활

BASE 8

팔진법 (Octal)

숫자: 0 ~ 7

접두사: 0 (C언어) 용도: UNIX 파일 권한

BASE 16

십육진법 (Hex)

숫자: 0~9, A~F

접두사: 0x (C언어) 용도: 메모리 주소, 색상

3-2 자릿값

자릿값 (Place Value) 원리

N진법에서 수 표현: $d_n \cdot r^n + d_{n-1} \cdot r^{n-1} + \dots + d_1 \cdot r^1 + d_0 \cdot r^0$

10진수 1234_{10} 분해

$$\begin{aligned} 1234 &= 1 \times 10^3 + 2 \times 10^2 + 3 \times 10^1 + 4 \times 10^0 \\ &= 1000 + 200 + 30 + 4 \end{aligned}$$

2진수 1011_2 분해

$$\begin{aligned} 1011_2 &= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\ &= 8 + 0 + 2 + 1 = 11_{10} \end{aligned}$$

16진수 $2AF_{16}$ 분해

$$\begin{aligned} 2AF_{16} &= 2 \times 16^2 + 10 \times 16^1 + 15 \times 16^0 \\ &= 512 + 160 + 15 = 687_{10} \end{aligned}$$

8진수 347_8 분해

$$\begin{aligned} 347_8 &= 3 \times 8^2 + 4 \times 8^1 + 7 \times 8^0 \\ &= 192 + 32 + 7 = 231_{10} \end{aligned}$$

10진	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
16진	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

3-3 이진법

이진법 (Binary) 심화

8비트 예: 1 0 1 1 0 1 0 1

$2^7=128$

$2^6=64$

$2^5=32$

$2^4=16$

$2^3=8$

$2^2=4$

$2^1=2$

$2^0=1$

계산

$128+0+32+16+0+4+0+1 = 181_{10}$

$\rightarrow 10110101_2 = 181_{10}$

 2^n 자주 쓰이는 값

$2^8=256$ $2^{10}=1024$ (1K)

$2^{16}=65536$ $2^{20}=1048576$ (1M)

$2^{32} \approx 4.3\text{억}$ $2^{64} \approx 1.8 \times 10^{19}$

```
// n번째 비트 값 확인
int bit_n = (num >> n) & 1;
```

```
// 10진 → 2진 출력 (재귀)
void print_bin(int n) {
    if (n > 1) print_bin(n / 2);
    printf("%d", n % 2);
}
```

8진수와 16진수 표현

8진수 (Octal)

숫자 0~7 사용

C: 0으로 시작 → $017 = 15_{10}$

자리당 3비트 대응

→ 2진수 그룹핑 편리

예) $011\ 111\ 010_2 = 172_8$

16진수 (Hexadecimal)

숫자 0~9, 문자 A~F 사용

C: 0x로 시작 → $0xFF = 255_{10}$

자리당 4비트(니블) 대응

→ 2진수 그룹핑 매우 편리

예) $1111\ 0010_2 = F2_{16}$

2진 (4비트)	8진	10진	16진
0000	0	0	0
0001	1	1	1
0010	2	2	2
0011	3	3	3
0100	4	4	4
0101	5	5	5
0110	6	6	6
0111	7	7	7
1000	10	8	8
1001	11	9	9
1010	12	10	A
1011	13	11	B
1100	14	12	C
1101	15	13	D
1110	16	14	E
1111	17	15	F

Section

04

진법 간 변환

10진수 ↔ 2·8·16진법 변환 알고리즘
단계별 나눗셈·곱셈 방법

10진수 → 2진수 변환 (정수부)

방법: 2로 반복 나누기 (나머지 역순 읽기)

- ① 수를 2로 나눈다
- ② 나머지를 기록한다 (0 또는 1)
- ③ 몫이 0이 될 때까지 반복
- ④ 나머지를 역순으로 읽으면 2진수

```
// 10진 → 2진 변환 코드
void dec_to_bin(int n) {
    if (n == 0) { printf("0"); return; }
    char bits[32]; int idx = 0;
    while (n > 0) {
        bits[idx++] = '0' + (n % 2);
        n /= 2;
    }
    for (int i = idx-1; i >= 0; i--)
        putchar(bits[i]);
}
```

예: $13_{10} \rightarrow 2진수$

나눗셈	몫	나머지
$13 \div 2$	6	1 ← LSB
$6 \div 2$	3	0
$3 \div 2$	1	1
$1 \div 2$	0	1 ← MSB

역순 읽기: 1 1 0 1 → 1101_2

검증: $1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 = 13$ ✓

2진수 → 10진수 변환

방법: 자릿값 × 비트 합산

각 자리 비트 × 2의 거듭제곱 값을 합산

가장 왼쪽 = 최상위 비트 (MSB)

가장 오른쪽 = 최하위 비트 (LSB)

// Horner 방법 (효율적)

// $1 \times 2 = 2$

// $2 \times 2 + 0 = 4$

// $4 \times 2 + 1 = 9$

// $9 \times 2 + 1 = 19$

// $19 \times 2 + 0 = 38 \leftarrow 100110_2 = 38_{10}$

```
int val = 0;
```

```
for each bit b (MSB→LSB): val = val*2 + b;
```

예: $10110_2 \rightarrow 10$ 진수

비트	자릿값	기여값
1 (4번째)	$2^4 = 16$	$1 \times 16 = 16$
0 (3번째)	$2^3 = 8$	$0 \times 8 = 0$
1 (2번째)	$2^2 = 4$	$1 \times 4 = 4$
1 (1번째)	$2^1 = 2$	$1 \times 2 = 2$
0 (0번째)	$2^0 = 1$	$0 \times 1 = 0$

합계: $16+0+4+2+0 = 22_{10}$

10진수 → 16진수 변환

방법: 16으로 반복 나누기

- ① 수를 16으로 나눈다
- ② 나머지 기록 (0~15 → 0~9, A~F)
- ③ 몫이 0이 될 때까지 반복
- ④ 나머지 역순 읽기

```
// 10진 → 16진 출력 (C)
printf("%X", 255); // FF
printf("%x", 255); // ff (소문자)
printf("0x%X", 255); // 0xFF
printf("%08X", 15); // 0000000F
```

예: $255_{10} \rightarrow 16진수$

나눗셈	몫	나머지
$255 \div 16$	15	$15 = F$
$15 \div 16$	0	$15 = F$

역순: F F $\rightarrow FF_{16} = 0xFF$

예: $687_{10} \rightarrow 16진수$

$687 \div 16$	42	$15 = F$
$42 \div 16$	2	$10 = A$
$2 \div 16$	0	2

역순: 2 A F $\rightarrow 2AF_{16}$

2진수 ↔ 16진수 빠른 변환

핵심 원리: 4비트 = 1 헥스 자리

2진수를 오른쪽부터 4비트씩 묶어서 16진수로 변환
16진수 한 자리를 4비트 2진수로 바로 변환 가능!

2진수 → 16진수

1011 1101 0010₂
= B D 2₁₆
= BD2₁₆ = 0xBD2

16진수 → 2진수

3F8A₁₆
= 0011 1111 1000 1010₂
= 0011111110001010₂

16진	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
2진	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

0xCAFE₁₆ = C A F E
= 1100 1010 1111 1110₂
= 1100101011111110₂

소수 부분 진법 변환

방법: 2를 반복 곱하기

- ① 소수 부분에 기수(2)를 곱한다
- ② 정수 부분을 기록한다 (0 또는 1)
- ③ 소수 부분이 0이 될 때까지 반복
- ④ 기록된 값을 순서대로 읽음 (위→아래)

예: $0.625_{10} \rightarrow 2진수$

소수부 $\times 2$	정수부 (기록)
$0.625 \times 2 = 1.25$	1 ← 첫 번째
$0.25 \times 2 = 0.5$	0
$0.5 \times 2 = 1.0$	1 ← 마지막

순환 소수 문제

$0.1_{10} = 0.0001100110011..._2$ (무한 순환)

컴퓨터는 유한 비트로 저장 → 부동소수점 오차 발생

예) $0.1 + 0.2 \neq 0.3$ (부동소수점 오차)

```
// 부동소수점 비교 (잘못된 방법)
if (0.1 + 0.2 == 0.3) { ... } // false!

// 올바른 방법 (epsilon 비교)
if (fabs((0.1 + 0.2) - 0.3) < 1e-9) { ... }
```

Section

05

진법별 사칙연산

2·8·16진법에서 직접 수행하는 사칙연산
올림수·빌림수 처리

5-1 2진 덧셈

2진수 덧셈

A	B	합 (Sum)	올림수 (Carry)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1 ← 올

반가산기 (Half Adder)

Sum = A XOR B ($A \oplus B$)Carry = A AND B ($A \& B$)

전가산기 (Full Adder)

Sum = $A \oplus B \oplus Cin$ Cout = $(A \& B) \mid (B \& Cin) \mid (A \& Cin)$ 예: $1011_2 + 1101_2$

$$\begin{array}{r} 1\ 0\ 1\ 1 \\ +\ 1\ 1\ 0\ 1 \\ \hline \end{array}$$

$$1\ 1\ 1$$

$$1\ 1\ 0\ 0\ 0 \quad (\text{올림수 표시})$$

$$= 11000_2 = 24_{10}$$

$$\text{검증: } 11 + 13 = 24 \quad \checkmark$$

// 2진수 덧셈 시뮬레이션

```
int add_bin(int a, int b) {
    int carry = 0, result = 0, bit = 1;
    while (a || b || carry) {
        int s = (a&1)^(b&1)^carry;
        carry =
            ((a&1)&(b&1)) | ((b&1)&carry) | ((a&1)&carry);
        result |= (s * bit);
        a>>=1; b>>=1; bit<<=1;
    }
    return result;
}
```

2진수 뺄셈과 2의 보수

2의 보수 (Two's Complement)

2의 보수 = 1의 보수 + 1

1의 보수: 각 비트를 반전

예) $5 = 0101 \rightarrow 1\text{의 보수: } 1010 \rightarrow 2\text{의 보수:}$

$1011 = -5$

$\Rightarrow$ 뺄셈: $A - B = A + (-B) = A + (2\text{의 보수}(B))$

직접 뺄셈 (빌림수 방법)

2진 뺄셈 규칙:

$0 - 0 = 0$

$1 - 0 = 1$

$1 - 1 = 0$

$0 - 1 = 1$, 다음 자리에서 빌림

예: $1001_2 - 0011_2$

① 0011 의 1의 보수: 1100

② 0011 의 2의 보수: $1100 + 1 = 1101$

③ $1001 + 1101 = 1,0110$ (올림수 버림)

④ 결과: $0110_2 = 6_{10}$

검증: $9 - 3 = 6 \quad \checkmark$

예: $1101_2 - 0101_2$

$1\ 1\ 0\ 1$

$- 0\ 1\ 0\ 1$

$1\ 0\ 0\ 0_2 = 8_{10}$

검증: $13 - 5 = 8 \quad \checkmark$

2진수 곱셈

원리: 십진수 필산과 동일

각 자리 비트로 부분곱(partial product)을 구하고
모두 더한다 (비트가 1이면 피승수 그대로, 0이면 0)

예: $1011_2 \times 101_2$ ($11 \times 5 = 55$)

1 0 1 1	(피승수)
× 1 0 1	(승수)
1 0 1 1	(× 1, 자리아동 0)
0 0 0 0	(× 0, 자리아동 1)
1 0 1 1	(× 1, 자리아동 2)
1 1 0 1 1 1 ₂	= 55 ₁₀ ✓

비트 시프트 활용

$\times 2$ = 왼쪽 1비트 시프트 ($\ll 1$)

$\times 4$ = 왼쪽 2비트 시프트 ($\ll 2$)

$\times 2^n$ = 왼쪽 n비트 시프트 ($\ll n$)

⇒ 빠른 곱셈 구현 가능

// 비트 시프트 곱셈

```
int fast_mul2(int x) { return x << 1; }
```

```
int fast_mul4(int x) { return x << 2; }
```

// 일반 2의 거듭제곱 곱셈

```
int fast_muln(int x, int n) {
    return x << n; // x × 2^n
}
```

// $7x = 8x - x = (x \ll 3) - x$

```
int mul7(int x) { return (x << 3) - x; }
```

2진수 나눗셈

원리: 오른쪽 시프트로 2 나눗셈

$\div 2$ = 오른쪽 1비트 시프트 ($\gg 1$)

$\div 4$ = 오른쪽 2비트 시프트 ($\gg 2$)

나머지는 최하위 비트로 판단 (홀수 여부)

예: $1011_2 \div 10_2$ ($11 \div 2$)

1 0 1 1 $\div$ 1 0

1 0 (부분 $\rightarrow$ 1 나머지 0)

0 1 $\downarrow$

0 1 1 (bring down 1)

0 1 0 $\rightarrow$ 1 나머지 1

몫: $101_2 = 5$, 나머지: $1_2 = 1$

검증: $11 = 2 \times 5 + 1$ ✓

// 비트 시프트 나눗셈

```
int div2(int x) { return x >> 1; }
```

```
int div4(int x) { return x >> 2; }
```

```
int div8(int x) { return x >> 3; }
```

// 나머지

```
int mod2(int x) { return x & 1; } // 홀수면 1
```

```
int mod4(int x) { return x & 3; }
```

```
int mod8(int x) { return x & 7; }
```

// 일반: $x \% (2^n) = x \& (2^n - 1)$

5-5 16진 연산

16진수 사칙연산

16진 덧셈

각 자리합이 16 이상이면 올림수 발생

예) $A + 7 = 11_{10} = 11$, 올림수 없음

$F + 1 = 16_{10} = 10_{16}$, 올림수 1

16진 뺄셈

빌림수 = 16

예) $3 - 9$: $3+16=19$, $19-9=10=A$, 빌림1

예: $5F3_{16} - 2A8_{16}$

자리0: $3-8 \rightarrow 3+16-8=11=B$, 빌림1

자리1: $F-A-1 = 15-10-1 = 4$

자리2: $5-2 = 3$

결과: $34B_{16}$

검증: $1523 - 680 = 843 = 0x34B$ ✓

예: $2B4_{16} + 1C9_{16}$

자리0: $4+9 = 13 = D \rightarrow D$, 올림0

자리1: $B+C = 11+12 = 23$
 $= 17_{16} \rightarrow 7$, 올림1

자리2: $2+1+1(\text{올림}) = 4$

결과: $47D_{16}$

검증: $692 + 457 = 1149 = 0x47D$ ✓

// 16진 출력 서식

`printf("%X", val);` // 대문자

`printf("%x", val);` // 소문자

`printf("%04X",val);` // 4 자리 0패딩

`printf("0x%02X",v);` // 0x 접두사

Section

06

컴퓨터에서의 수의 표현

부호 비트 · 2의 보수 · 부동소수점
IEEE 754 · 오버플로우 · 정밀도 문제

6-1 부호 표현

부호 있는 정수 표현 방법

부호-크기 (Sign-Magnitude)

 $+5 = 0\ 0000101$ $-5 = 1\ 0000101$

[최상위 비트 = 부호]

문제: +0과 -0 두 개 존재

1의 보수 (One's Complement)

 $+5 = 0\ 0000101$ $-5 = 1\ 1111010$ (비트 반전)

문제: +0, -0 여전히 두 개

현재 거의 사용 안 함

2의 보수 (Two's Complement)

 $+5 = 0\ 0000101$ $-5 = 1\ 1111011$ (반전+1)

장점: 덧셈/뺄셈 회로 단순

현재 표준 방식!

4비트 표현	부호-크기	1의 보수	2의 보수 ★
+7	0111	0111	0111
+1	0001	0001	0001
+0	0000	0000	0000
-0	1000	1111	없음
-1	1001	1110	1111
-7	1111	1000	1001
-8	없음	없음	1000

2의 보수 (Two's Complement) 상세

$$-n = \sim n + 1 \quad (\text{비트 반전 후 1 더하기})$$

8비트 예시:

+13 = 0 0 0 0 1 1 0 1

NOT = 1 1 1 1 0 0 1 0 (1의 보수)

+1 =

-13 = 1 1 1 1 0 0 1 1 (2의 보수)

검증: 0 0 0 0 1 1 0 1 (+13)

+ 1 1 1 1 0 0 1 1 (-13)

= 1 0 0 0 0 0 0 0 (올림수 버림 = 0

✓)

n비트 2의 보수 범위

양수 최대: $2^{(n-1)} - 1$

음수 최소: $-2^{(n-1)}$

8비트: -128 ~ +127

16비트: -32768 ~ +32767

32비트: -2,147,483,648 ~ +2,147,483,647

특수한 음수: 최솟값

-128 (8비트) = 1000 0000

이 값의 2의 보수 = 1000 0000 (자기 자신!)

→ 절댓값을 양수로 표현 불가

→ abs(-128) 사용 시 오버플로우 주의!

6-3 부동소수점

부동소수점 (Floating Point) 개요

부호 $\times$ 가수(Mantissa) $\times$ 기수^지수

S	지수부 (8비트)	가수부 (23비트)
부호 1비트	지수+127 (bias=127)	소수점 이하 (1. 생략)

예: $13.5_{10} \rightarrow$ IEEE 754 단정도

- ① $13.5_{10} = 1101.1_2$
- ② 정규화: 1.1011×2^3
- ③ 지수: $3 + 127 = 130 = 10000010_2$
- ④ 가수: 10110000000000000000000

결과: 0 10000010 10110000000000000000000
= 0x41580000

6-4 IEEE 754

IEEE 754 표준

형식	총 비트	부호	지수부	가수부	Bias	정밀도 (유효 자릿수)
float (단정도)	32	1	8	23	127	약 7자리
double (배정도)	64	1	11	52	1023	약 15~16자리
long double	80/128	1	15	63/112	16383	약 18~19자리

0

지수=0, 가수=0 (부호 있는 0)

비정규화 수

지수=0, 가수≠0 (0에 가까운 작은 수)

무한대 $\pm\infty$

지수=모두 1, 가수=0

NaN

지수=모두 1, 가수≠0 (Not a Number)

```
float f = 1.0f / 0.0f;    // +Infinity
float n = 0.0f / 0.0f;    // NaN
if (isnan(n)) printf("NaN!");
if (isinf(f)) printf("Infinity!");
```

6-5 부동소수점 오차

부동소수점 오차와 정밀도

표현 오차 (Representation Error)

$0.1_{10} = 0.0001100110011\dots_2$ (무한 순환)

컴퓨터에서 정확히 표현 불가 → 반올림 오차 발생

소거 오차 (Cancellation Error)

거의 같은 두 수를 뺄 때 유효 자릿수 손실

예) $1.0000001 - 1.0000000 = 0.0000001$

→ 상대 오차 매우 커질 수 있음

```
// 전형적인 부동소수점 오차 예시
#include <stdio.h>
int main() {
    printf("%.17f\n", 0.1 + 0.2);
    // 출력: 0.30000000000000004
    printf("%d\n", 0.1 + 0.2 == 0.3);
    // 출력: 0 (false!)

    // 올바른 비교
    double eps = 1e-9;
    if (fabs(0.1 + 0.2 - 0.3) < eps)
        printf("같다!\n"); // 이 방법 사용!
}
```

6-6 데이터 타입

C언어 정수형 데이터 타입

타입	크기	범위	최댓값 상수
char	1 byte	-128 ~ 127	CHAR_MAX
unsigned char	1 byte	0 ~ 255	UCHAR_MAX
short	2 bytes	-32,768 ~ 32,767	SHRT_MAX
unsigned short	2 bytes	0 ~ 65,535	USHRT_MAX
int	4 bytes	-2,147,483,648 ~ 2,147,483,647	INT_MAX
unsigned int	4 bytes	0 ~ 4,294,967,295	UINT_MAX
long long	8 bytes	$-9.2 \times 10^{18} \sim +9.2 \times 10^{18}$	LLONG_MAX
unsigned long long	8 bytes	$0 \sim 1.8 \times 10^{19}$	ULLONG_MAX

```
#include <limits.h>
printf("%d", INT_MAX);      // 2147483647
printf("%lld", LLONG_MAX); // 9223372036854775807
printf("%u", UINT_MAX);    // 4294967295
```

BCD 코드 (Binary Coded Decimal)

BCD란?

10진수 각 자리를 4비트 2진수로 표현
계산기·디스플레이에서 정확한 10진수 표현에 사용
금융 계산: 0.1 오차 없이 처리 가능

예: 937_{10} BCD 표현

9 3 7
1001 0011 0111
= 1001 0011 0111 BCD
(24비트로 3자리 표현)

BCD 덧셈 규칙

합이 9 이하: 그대로
합이 10~15: + 0110_2 (6) 보정
합이 16 이상: 올림수 발생, 나머지 + 0110_2 보정

10진	BCD (4비트)	10진	BCD
0	0000	5	0101
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

6-8 문자 코드

문자 코드: ASCII · UTF

ASCII (American Standard Code for Information Interchange)

7비트로 128개 문자 표현 (0~127)

영문 대소문자, 숫자, 특수문자, 제어문자 포함

'A'=65, 'a'=97, '0'=48

문자	10진	16진	2진	문자	10진	16진
'0'	48	0x30	00110000	'A'	65	0x41
'1'	49	0x31	00110001	'B'	66	0x42
'9'	57	0x39	00111001	'Z'	90	0x5A
'a'	97	0x61	01100001	' '(공백)	32	0x20
'z'	122	0x7A	01111010	'\n' (줄바꿈)	10	0x0A

'A' + 32 = 'a' (대→소문자)

'a' - 32 = 'A' (소→대문자)

'5' - '0' = 5 (문자 숫자→정수)

isdigit('3') → true

isupper('A') → true

6-9 비트 마스킹

비트 마스킹 (Bit Masking) 기법

특정 비트 ON

```
n |= (1 << k); // k번째 비트 1로 설정
```

특정 비트 OFF

```
n &= ~(1 << k); // k번째 비트 0으로 설정
```

특정 비트 토글

```
n ^= (1 << k); // k번째 비트 반전
```

특정 비트 확인

```
(n >> k) & 1 // k번째 비트 값 (0 또는 1)
```

```
// 실전 예: 플래그 관리
#define FLAG_A 0x01 // 0000 0001
#define FLAG_B 0x02 // 0000 0010
#define FLAG_C 0x04 // 0000 0100

int status = 0;
status |= FLAG_A; // A 켜기
status |= FLAG_B | FLAG_C; // B,C 켜기
status &= ~FLAG_B; // B 끄기
if (status & FLAG_C) ... // C 확인
```

부호 확장 (Sign Extension)

부호 확장이란?

적은 비트 수의 정수를 더 많은 비트로 변환할 때
부호(양수/음수)를 유지하는 방법
양수: 상위 비트에 0을 채움
음수: 상위 비트에 1을 채움 (부호 비트 복사)

```
// C언어 자동 부호 확장
char c = -50;           // 8비트: 1100 1110
int i = c;              // 32비트: 자동 확장
printf("%d", i);       // -50 (올바름)
```

```
// 주의: unsigned char는 zero extension
unsigned char uc = 200; // 1100 1000
int ui = uc;
printf("%d", ui);      // 200 (양수 그대로)
```

8비트 → 16비트 부호 확장:

```
+50 = 0011 0010 (8비트)
    = 0000 0000 0011 0010 (16비트)
      ↑ 0으로 채움
```

```
-50 = 1100 1110 (8비트, 2의 보수)
    = 1111 1111 1100 1110 (16비트)
      ↑ 1로 채움 (부호 비트=1 복사)
```

타입 캐스팅 주의

```
int → char: 하위 8비트만 남음 (절삭)
char → int: 부호 확장 자동
int i = 300; char c = (char)i; → c = 44
```

6-11 오버플로우

오버플로우 심화 분석

```
// 오버플로우 발생 케이스들
int a = INT_MAX;           // 2147483647
int b = a + 1;             // 오버플로우! → -2147483648

unsigned int u = 0;
u = u - 1;                 // 언더플로우! → 4294967295

int c = INT_MIN;
int d = -c;                // 오버플로우! (2^31은 int 범위 초과)

int big = 100000;
int sq = big * big;         // 오버플로우! 10^10 > INT_MAX
long long sq2 = (long long)big * big; // 안전!
```

덧셈 오버플로우 검출

```
if (b > 0 && a > INT_MAX - b) overflow;
if (b < 0 && a < INT_MIN - b) overflow;
```

곱셈 오버플로우 검출

```
if (a != 0 && b > INT_MAX / a) overflow;
(또는 long long으로 계산 후 범위 확인)
```

KOI 대회 팁

두 수 곱셈 전 long long 캐스팅!
모듈러 덧셈 시 m 더한 후 % m 처리
INT_MAX + 1 연산 절대 금지

⚠ 오버플로우는 C에서 미정의 동작(UB) — 절대 의존 금지!

컴퓨터 메모리 구조와 엔디안

빅 엔디안 (Big Endian)

MSB(최상위 바이트) 먼저 저장

예) 0x12345678 → 12 34 56 78

네트워크 프로토콜에서 주로 사용

리틀 엔디안 (Little Endian)

LSB(최하위 바이트) 먼저 저장

예) 0x12345678 → 78 56 34 12

x86/x86-64 (PC) 에서 사용

```
// 엔디안 확인 코드
int n = 1;
char *p = (char*)&n;
if (*p == 1) printf("Little Endian\n");
else      printf("Big Endian\n");

// 0x12345678 메모리 배치
// Little: 주소 0x00: 78  0x01: 56  0x02: 34  0x03: 12
// Big:    주소 0x00: 12  0x01: 34  0x02: 56  0x03: 78
```

6-13 비트 심화

비트 조작 심화: 트릭과 최적화

최하위 1비트 추출 $n \& (-n)$ $= n \& (\sim n + 1)$, 쓸모 있는 비트 분리

최하위 1비트 제거 $n \& (n-1)$ 반복 시 1비트 개수 세기 $O(\text{비트수})$

부호 반전 없이 절댓값 `int m=n>>(BITS-1); (n+m)^m` branch-free 절댓값

두 값 swap (XOR) $a^=b; b^=a; a^=b;$ 임시 변수 불필요 (같은 주소 시 주의)

2의 거듭제곱 판별 $n > 0 \&\& !(n \& (n-1))$ n 이 2의 거듭제곱이면 true

```
// popcount: 1비트 개수 세기
int popcount(int n) {
    int cnt = 0;
    while (n) { n &= (n-1); cnt++; }
    return cnt;
}
// 또는 builtin_popcount(n); (GCC)
```

6-14 FP 비교

부동소수점 정밀도 문제 실전

```
// 문제 1: 누적 오차
double sum = 0.0;
for (int i = 0; i < 1000000; i++)
    sum += 0.1;
printf("%.6f\n", sum); // 100000.007629 (오차!)

// 문제 2: 비교 오차
double a = 1.0 / 3.0;
double b = a * 3.0;
if (b == 1.0) printf("맞음"); // 출력 안 될 수도!

// 해결: 상대 오차 비교
double eps = 1e-9;
if (fabs(b - 1.0) < eps) printf("맞음"); // 안전!

// 경쟁 프로그래밍: 오차 허용 범위 확인!
// 문제에서 '절대/상대 오차 1e-6 이하' 등 명시
```

Kahan 합산 알고리즘

부동소수점 누적 오차 최소화 기법

```
double sum=0, c=0;
for each x: y=x-c; t=sum+y; c=(t-sum)-y; sum=t;
```

Section 06 핵심 정리

2의 보수

현대 컴퓨터 음수 표현 표준. $-n = \sim n + 1$

IEEE 754

float=32bit, double=64bit. 부호+지수+가수

오버플로우

int 범위 초과 시 UB. long long 또는 모듈러 사용

부동소수점 오차

$0.1 + 0.2 \neq 0.3$. epsilon 비교 사용

비트 마스크

$\&, ^, \sim, <, >$ 활용. 플래그·서브셋 처리

엔디안

x86 = 리틀 엔디안. 네트워크 = 빅 엔디안

Chapter 01 개념 연결 맵

수의 체계
 $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$

수의 표현과 연산

진법별 변환
2·8·10·16

진법 사칙연산
2진 $+$ $-$ $\times$ $\div$

수의 연산
사칙·비트·모듈러

진법 간 변환
나눗셈/곱셈법

컴퓨터 표현
IEEE754·보수

KOI 기출 유형 및 출제 패턴

진법 변환 직접 계산

2진수 ↔ 16진수 변환, N진법으로 표현된 수 계산

오버플로우 처리

큰 수 곱셈 long long 처리, 모듈러 산술 적용

소수 판별/에라토스테네스

소수 목록 생성, 소인수분해, N 이하 소수 개수

비트 연산 활용

서브셋 열거 ($1 \leq N$), 비트마스크 DP, 비트 카운팅

GCD/LCM 응용

유클리드 호제법, 여러 수의 최대공약수·최소공배수

모듈러 산술

큰 수 나머지, 빠른 거듭제곱, 페르마 소정리 응용

연습 문제 ①

Q
110110111₂를 10진수로 변환하시오.

힌트: 자릿값 × 비트 합산

Q
2183₁₀를 16진수로 변환하시오.

힌트: 16으로 반복 나눔

Q
33F8₁₆를 2진수로 변환하시오.

힌트: 각 16진 자리 → 4비트

Q
4101101₂ + 011011₂를 2진수로 계산하시오.

힌트: 자리별 덧셈, 올림수 처리

Q
5

gcd(84, 36)을 유클리드 호제법으로 구하시오.

힌트: 84=2×36+12, 36=3×12+0 ...

Q
6

-37의 8비트 2의 보수 표현을 구하시오.

힌트: +37 표현 후 비트 반전+1

답안

연습 문제 ① 답안

A
1

$$10110111_2 = 128+32+16+4+2+1 = 183_{10}$$

A
2

$$\begin{aligned} 183 \div 16 &= 11 \text{ 나머지 } 7 \\ 11 \div 16 &= 0 \text{ 나머지 } 11=B \\ &\rightarrow B7_{16} \end{aligned}$$

A
3

$$\begin{aligned} 3 &= 0011, F=1111, 8=1000 \\ &\rightarrow 0011 \ 1111 \ 1000_2 \end{aligned}$$

A
4

$$\begin{array}{r} 101101 \\ + 011011 \\ \hline 1001000_2 = 72_{10} \end{array}$$

A
5

$$\begin{aligned} 84 &= 2 \times 36 + 12 \\ 36 &= 3 \times 12 + 0 \\ \therefore \text{gcd} &= 12 \end{aligned}$$

A
6

$$\begin{aligned} +37 &= 0010 \ 0101 \\ 1 \text{의 보수} &= 1101 \ 1010 \\ +1 &= 1101 \ 1011 \\ \rightarrow -37 &= 1101 \ 1011_2 \end{aligned}$$

심화 연습 문제 ②

1. 1,000,000,000 (10억)을 2진수로 표현할 때 몇 비트가 필요한가?
(힌트: $2^{30} \approx 10^9$)
2. $N = 1234567890$ 에 대해 $N \bmod 1000000007$ 을 빠른 거듭제곱 없이
오버플로우 없이 계산하는 방법을 코드로 작성하시오.
3. 8비트 부호 없는 정수 두 개의 합이 255를 초과하는지 검사하는
오버플로우 없는 방법을 비트 연산만으로 작성하시오.
4. IEEE 754 단정도에서 표현 가능한 가장 작은 양의 정규화 수는?
(힌트: 지수부 = 00000001, 가수부 = 0...0)
5. 비트 마스킹으로 1~N까지 모든 부분집합(서브셋)을 열거하는
코드를 작성하시오. ($N \leq 20$ 가정)

C언어 수 처리 Quick Reference

```
/* — 진법 출력 — */  
printf("%d", n);           // 10진  
printf("%o", n);           // 8진  
printf("%x", n);           // 16진 소문자  
printf("%X", n);           // 16진 대문자  
printf("%b", n);           // 2진 (비표준, GCC 확장)
```

```
/* — 비트 연산 — */  
n & (1<<k)                  // k번째 비트 확인  
n | (1<<k)                  // k번째 비트 ON  
n & ~(1<<k)                 // k번째 비트 OFF  
n ^ (1<<k)                  // k번째 비트 토글  
__builtin_popcount(n)      // 1비트 개수 (GCC)  
__builtin_clz(n)            // 선행 0비트 개수
```

```
/* — 오버플로우 방지 — */  
(long long)a * b           // 곱셈 전 캐스팅  
((a % M) + (b % M)) % M    // 모듈러 덧셈  
(long long)a*b % M         // 모듈러 곱셈
```

핵심 알고리즘 모음

```
// 1. 유클리드 호제법
int gcd(int a,int b){return b?gcd(b,a%b):a;}
int lcm(int a,int b){return a/gcd(a,b)*b;}

// 2. 소수 판별
bool prime(int n){if(n<2)return 0;
    for(int i=2;i*i<=n;i++)if(n%i==0)return 0;return 1;}

// 3. 빠른 거듭제곱
ll pw(ll a,ll b,ll m){ll r=1;a%=m;
    for(;b>=1){if(b&1)r=r*a%m;a=a*a%m;}return r;}

// 4. 에라토스테네스의 체
bool sieve[MAX]; // false= 소수
void make_sieve(){sieve[0]=sieve[1]=1;
    for(int i=2;i*i<MAX;i++)
        if(!sieve[i])for(int j=i*i;j<MAX;j+=i)sieve[j]=1;}

// 5. 10진→2진 문자열
string to_bin(int n){string s="";
    do{s=(char)('0'+n%2)+s;n/=2;}while(n);return s;}
```

Chapter 01 완료

수의 표현과 연산

- ✓ 수의 체계와 포함 관계
- ✓ 사칙연산·비트연산·모듈러
- ✓ 2·8·16진법 변환 알고리즘
- ✓ 진법별 사칙연산
- ✓ 2의 보수·부동소수점
- ✓ 비트 마스크링 테크닉

NEXT →

Chapter 02 논리와 명제

- 명제 논리 (Propositional Logic)
- 진리표와 논리 연산자
- 논리 동치와 드모르간 법칙
- 술어 논리 (Predicate Logic)
- 추론 규칙과 증명 방법